COMP9334 Capacity Planning for Computer Systems and Networks

Week 5B_2: Discrete event simulation (4): Generating random numbers

This lecture

- Discrete event simulation
  - Week 4B: How to structure the simulation
  - Weeks 5A, 5B_1: Statistical analysis
- This lecture
  - Background on random numbers and how they are generated
  - How to generate random numbers of any probability distribution
  - Reproducibility
- Motivation
  - The Python random library can generate random numbers from many probability distributions but sometimes you may need a distribution that the library does not have

Random number generator in C

- In C, the function *rand()* generates random integers between 0 and RAND_MAX
- E.g. The following program generates 10 random integers:

```
#include <stdio.h>
#include <stdlib.h>

int main ()
{
  int i;

   for (i = 0; i < 10; i++)
    printf("%d\n",rand());

  return;

}
```

Let us generate 10,000 random integers using rand() and see how they are distributed

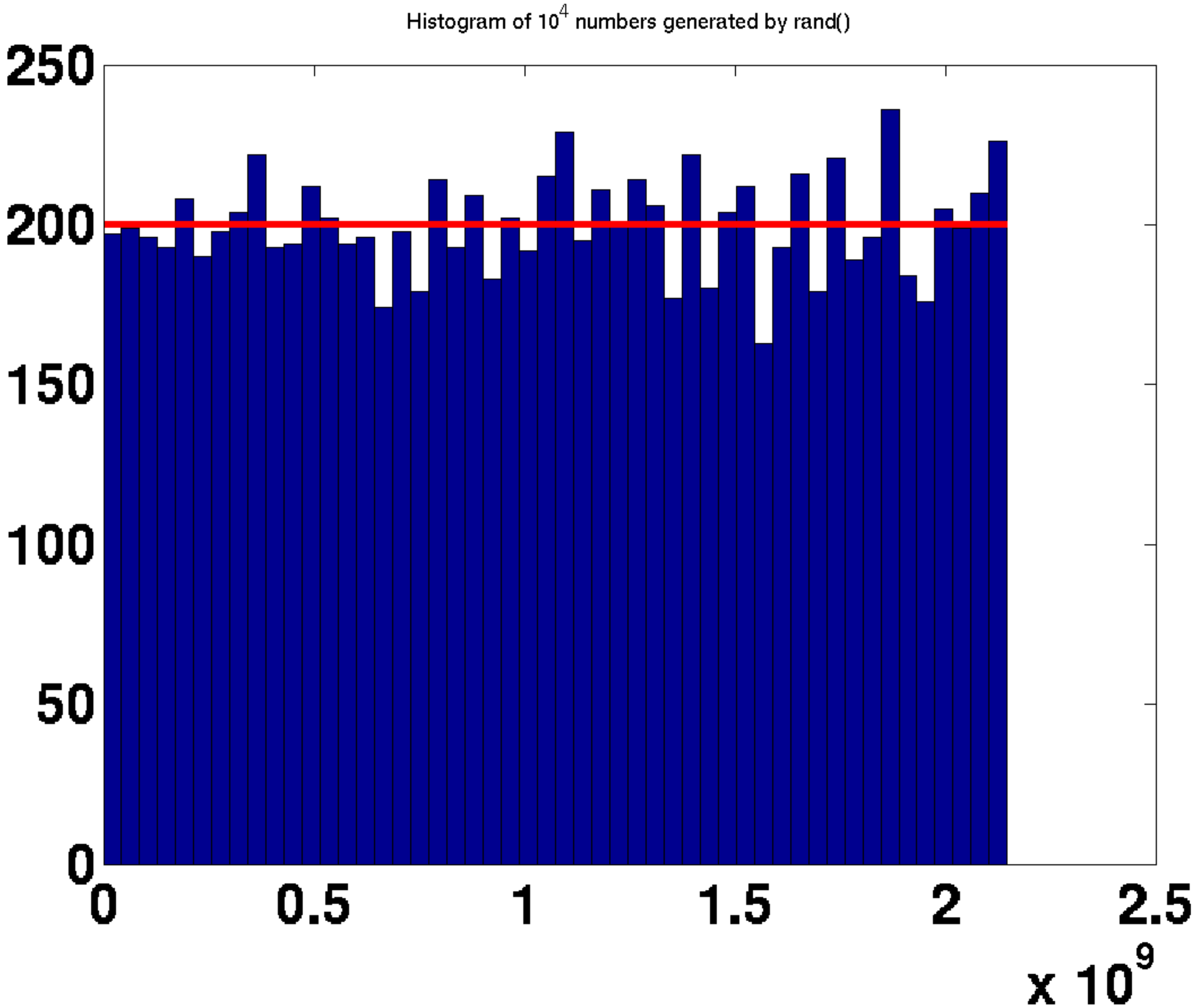
This C file “genrand1.c” is available from the course web site.

Distribution of 10000 entries from rand()

Sort into 50 bins

If the numbers are really uniformly distributed, we expect 200 numbers in each bin.

The numbers are almost uniformly distributed



LCG

- The random number generator in C is a Linear Congruential Generator (LCG)
- LCG generates a sequence of integers $\{Z_1, Z_2, Z_3, \ldots\}$ according to the recursion

  $$Z_k = a\, Z_{k-1} + c \pmod{m}$$

  where *a, c* and *m* are integers
- By choosing a, c, m, Z_1 appropriately, we can obtain a sequence of seemingly random integers
- If a = 3, c = 0, m = 5, Z_1 = 1, LCG generates the sequence 1, 3, 4, 2, 1, 3, 4, 2, …
- *Fact:* The sequence generated by LCG has a cycle of *m-1*
- We must choose *m* to be a large integer
  - For C, $m = 2^{31}$
- The proper name for the numbers generated is *pseudo-random numbers*

Seed

- LCG generates a sequence of integers $\{Z_1, Z_2, Z_3, \ldots\}$ according to the recursion

  $$Z_k = a\, Z_{k-1} + c \pmod m$$

  where *a, c* and *m* are integers
- The term Z_1 is call a seed
- By default, C also uses 1 as the seed and it will generate the same random sequence
- However, sometimes you need to generate different random sequences and you can change the seed by calling the function *srand()* before using *rand()*
  - Demo genrand1.c, genrand2.c and genrand3.m
  - *genrand1.c – uses the default seed*
  - *genrand2.c – sets the seed using command line argument*
  - *genrand3.c – sets the seed using current time*

Uniformly distributed random numbers between (0,1)

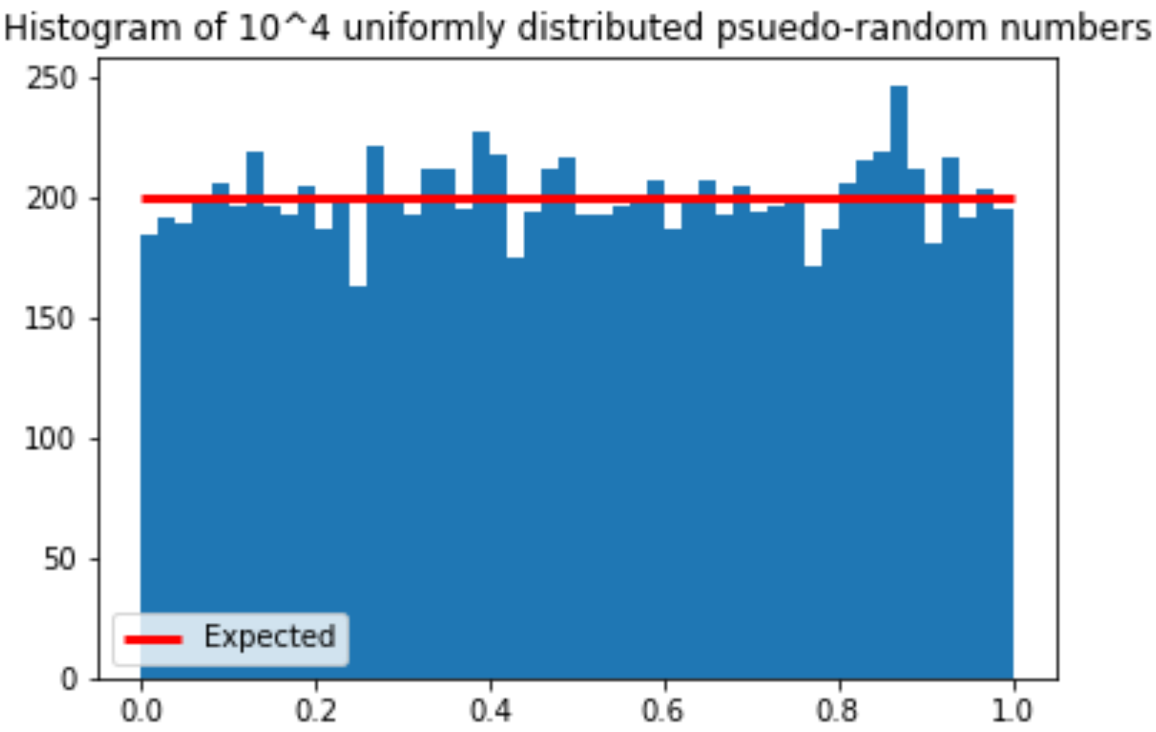
- With rand() in C, you can generate uniformly distributed random numbers in between 1 and 2^{31}-1(= RAND_MAX)
  - By dividing the numbers by RAND_MAX, you get randomly distributed numbers in (0,1)

- In Python, uniformly distributed random numbers in (0,1) can be generated by random.random() or numpy.random.random()
  - Both libraries uses the Mersenne Twister random number generator with a period of 2^{19937} - 1
  - If you use 10^9 random number in a second, the sequence will only repeat after 10^{5985} years

- Why are uniformly distributed random numbers important?
  - If you can generate uniformly distributed random numbers between (0,1), you can generate random numbers for any probability distribution

Random numbers generated by numpy

- 10,000 numbers generated by numpy.random.random()
  - Code in rand_uni.py

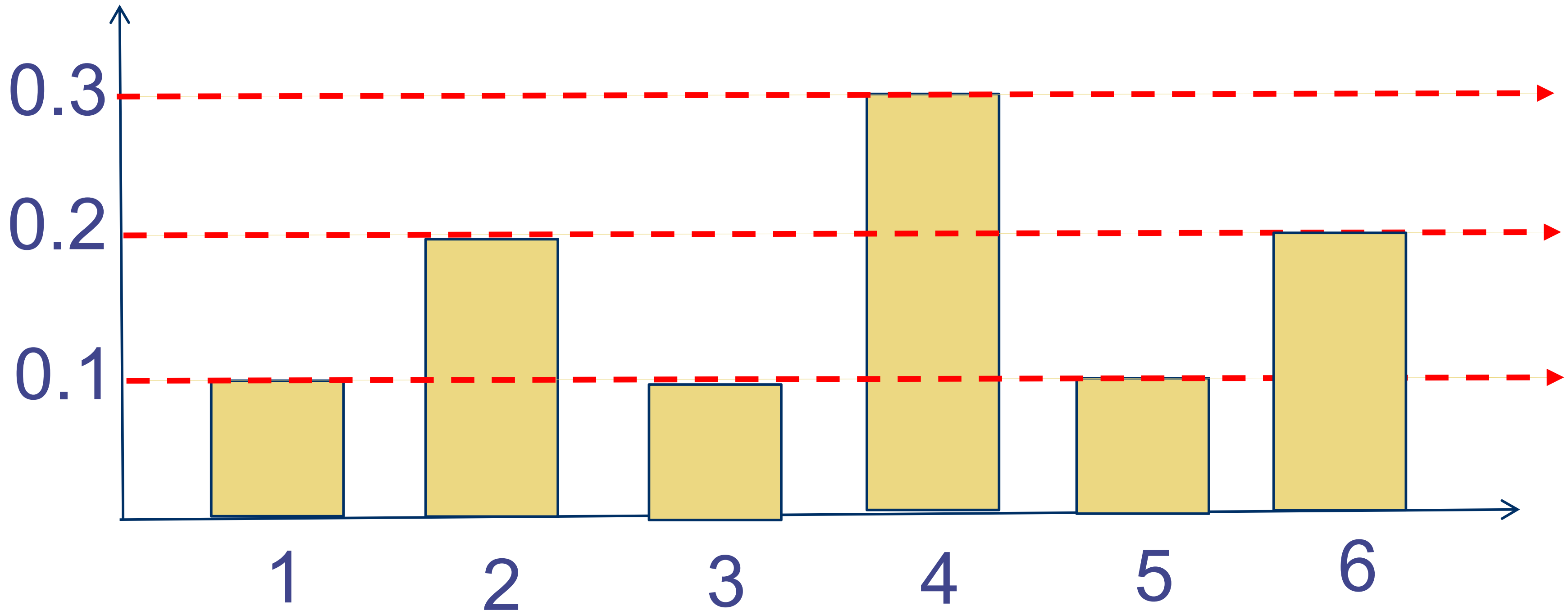


Fair coin distribution

- You can generate random numbers between 0 and 1
- You want to use these random numbers to imitate fair coin tossing, i.e.
  - Probability of HEAD = 0.5
  - Probability of TAIL = 0.5
- You can do this using the following algorithm
  - Generate a random number *u*
  - If u < $\boxed{}$, output HEAD
  - If u ≥ $\boxed{}$, output TAIL

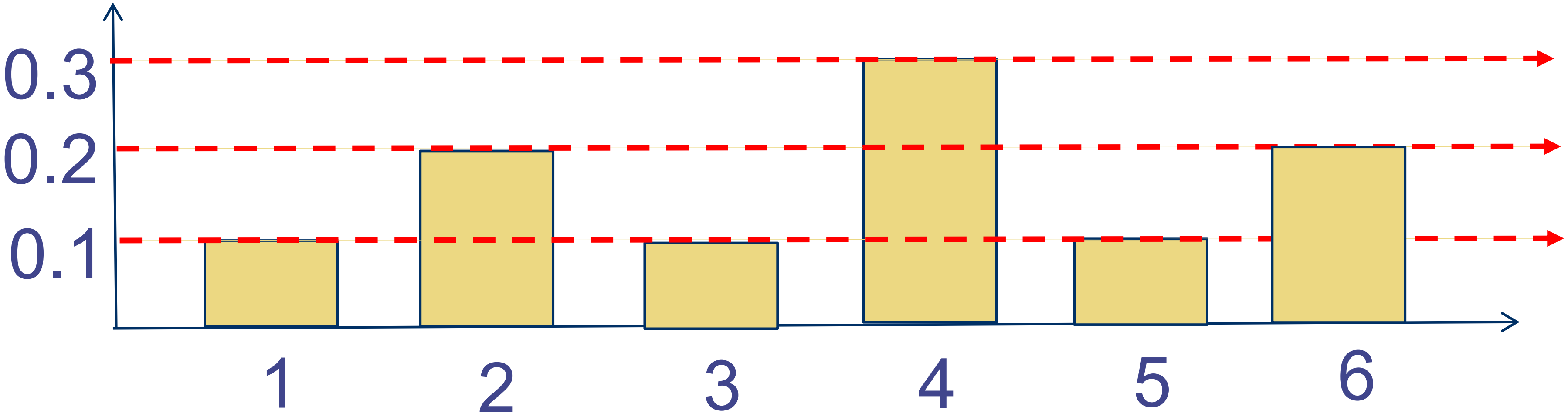
A loaded die

- You want to create a loaded die with probability mass function



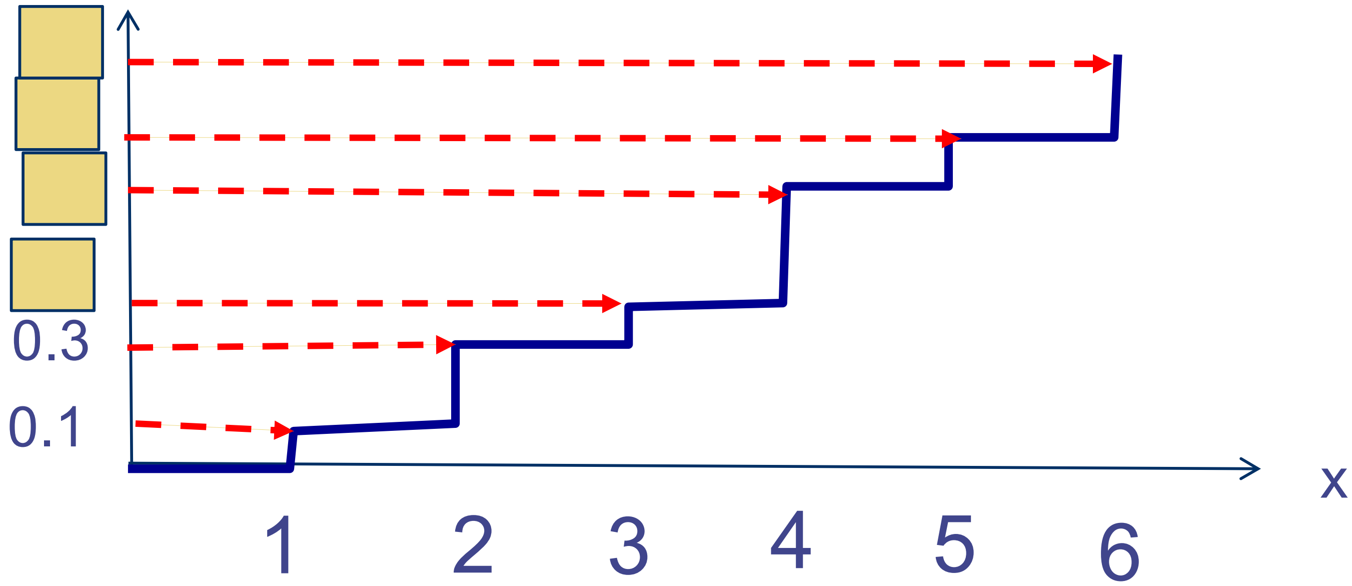
- The algorithm is:
  - Generate a random number u
  - If u < 0.1, output 1
  - If ≤ u < , output 2
  - If ≤ u < , output 3
  - If ≤ u < , output 4
  - If ≤ u < , output 5
  - If ≤ u , output 6

Cumulative probability distribution



Probability that the die gives a value ≤ x

Ex: Can you work out what these levels should be

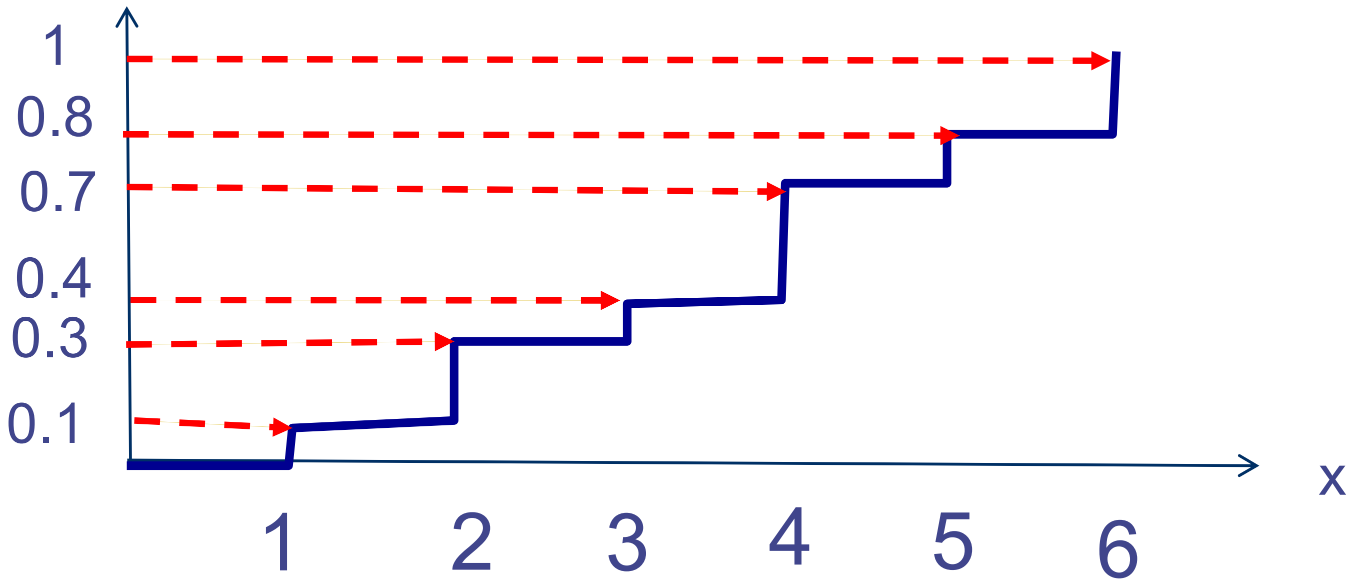


Comparing algorithm with cumulative distribution

- The algorithm is:
  - Generate a random number u
  - If u < 0.1, output 1
  - If 0.1 ≤ u < 0.3, output 2
  - If 0.3 ≤ u < 0.4, output 3
  - If 0.4 ≤ u < 0.7, output 4
  - If 0.7 ≤ u < 0.8, output 5
  - If 0.8 ≤ u , output 6

Probability that the die gives a value ≤ x

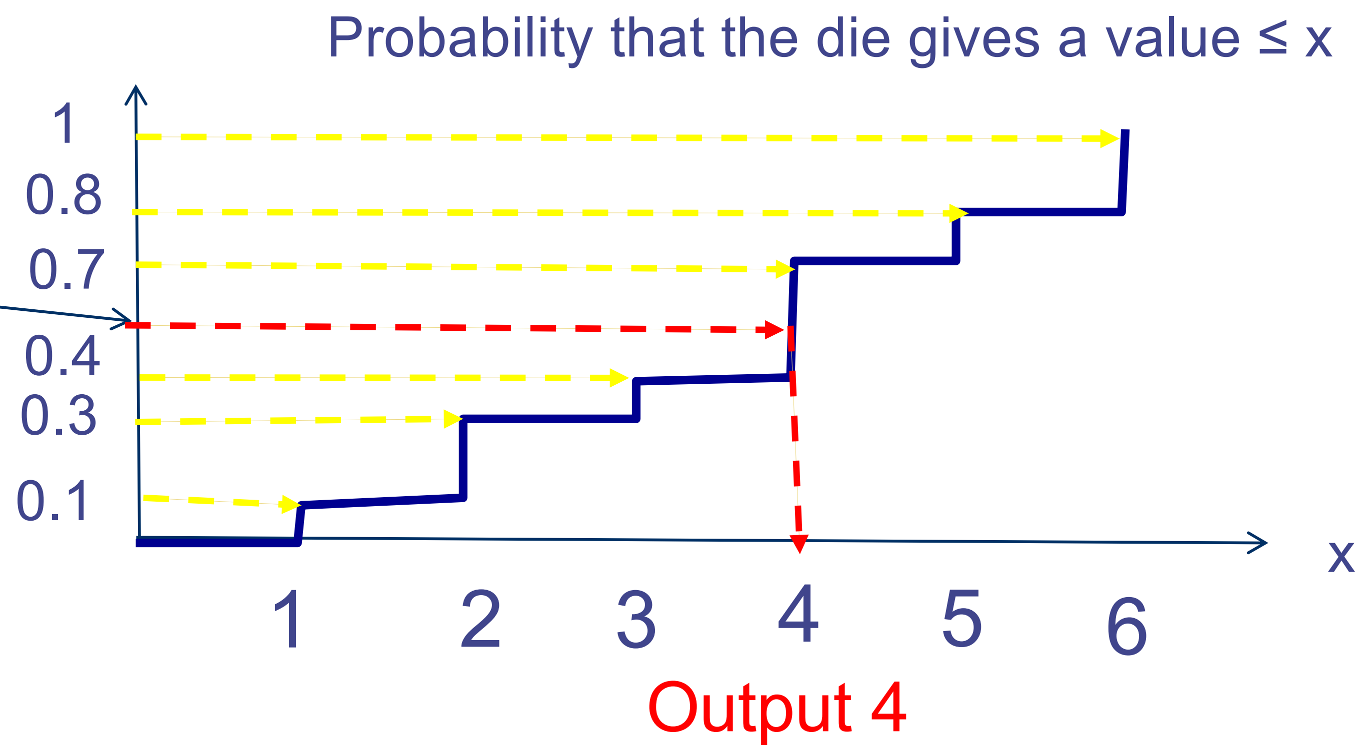
Ex: What do you notice about the intervals in the algorithm and the cumulative distribution?



Graphical interpretation of the algorithm

- The algorithm is:
  - Generate a random number u
  - If $u < 0.1$, output 1
  - If $0.1 \leq u < 0.3$, output 2
  - If $0.3 \leq u < 0.4$, output 3
  - If $0.4 \leq u < 0.7$, output 4
  - If $0.7 \leq u < 0.8$, output 5
  - If $0.8 \leq u$, output 6

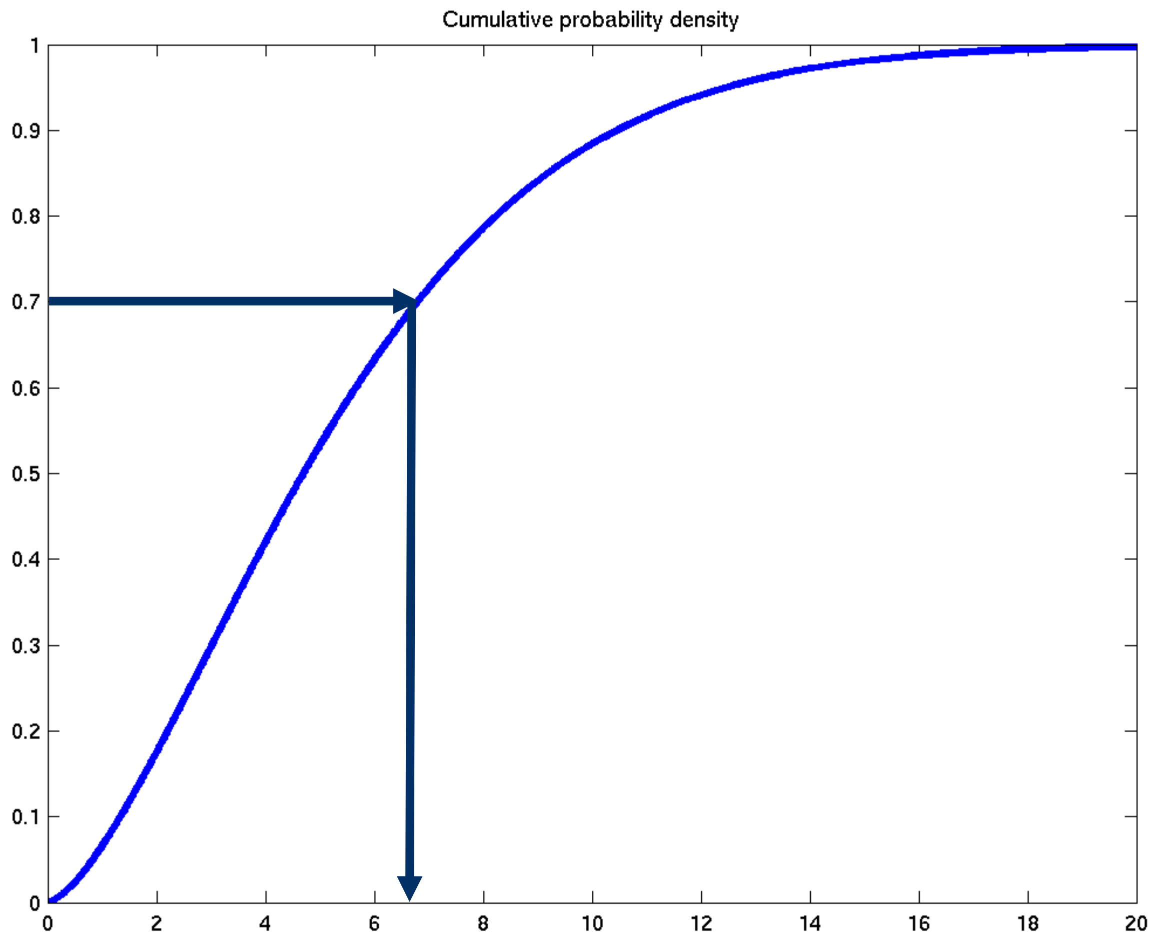
Ex: Let us assume u = 0.5126, what should the algorithm output?



Graphical representation of inverse transform method

- Consider the cumulative density function (CDF) y = *F(x)*, showed in the figure below

For this particular F(x), if u = 0.7 is generated then $F^{-1}(0.7)$ is 6.8



Inverse transform method

- A method to generate random number from a particular distribution is the *inverse transform method*

- In general, if you want to generate random numbers with cumulative density function (CDF) $F(x) = Prob[X \leq x]$, you can use the following procedure:
  - Generate a number u which is uniformly distributed in (0,1)
  - Compute the number $F^{-1}(u)$

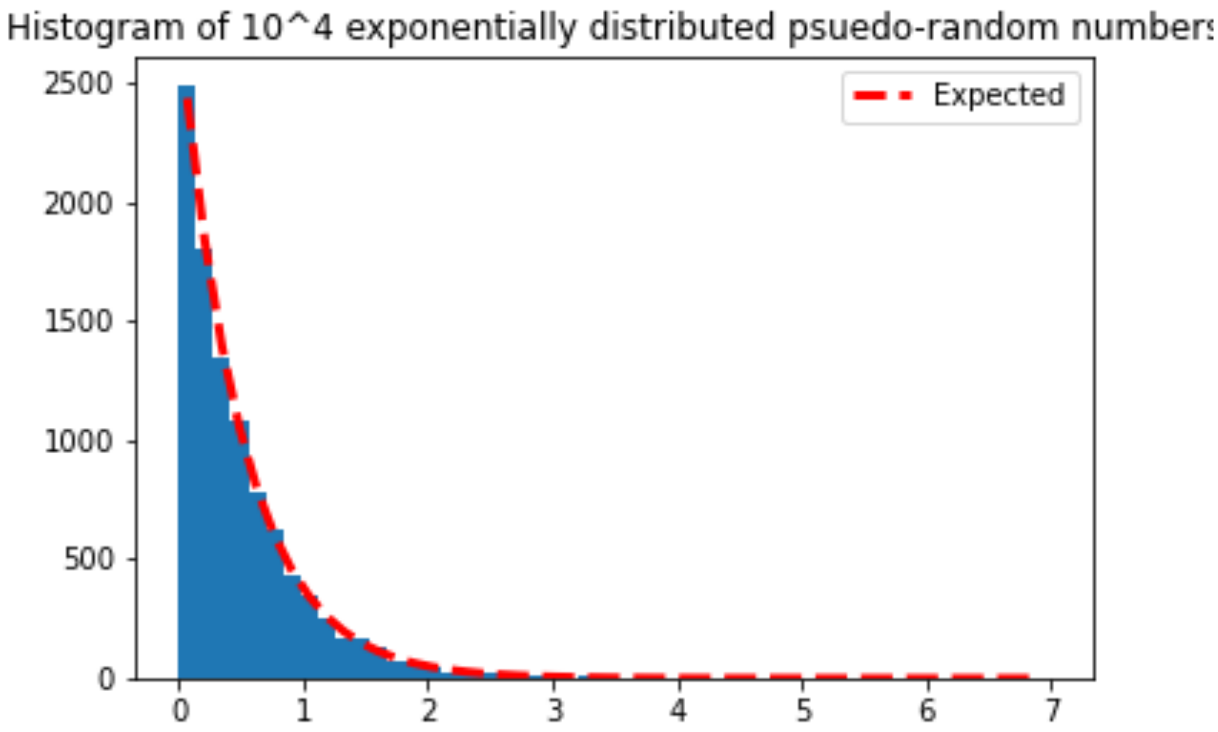
- Example: Let us apply the inverse transform method to the exponential distribution
  - *CDF is* $1 - \exp(-\lambda x)$

Generating exponential distribution

- Given a sequence $\{U_1, U_2, U_3, \ldots\}$ which is uniformly distributed in (0,1)
- The sequence $-\log(1 - U_k)/\lambda$ is exponentially distributed with rate λ

- (Python file hist_expon.py)

1. Generate 10,000 uniformly distributed numbers in (0,1)
2. Compute $-\log(1-u_k)/2$ where u_k are the numbers generated in Step 1
3. The plot shows
   1. The histogram of the numbers generated in Step 2 in 50 bins
   2. The red line show the expected number of exponential distributed numbers in each bin



Reproducible simulation – motivation

- You may recall that when we run the simulation sim_mm1.py, each simulation run gives a different result because different set of random numbers is used
- Doing simulation is like performing a scientific experiment
- Good science demands reproducibility
  - E.g., If you claim that the mean response time of a simulation run is say 1.3579, other people should be able to reproduce your result

Reproducible simulation

- In order to realise reproducibility of results, you need to save the state of the random number generator before simulation If you reuse the setting later, you can reproduce the result
  - The state of the Mersenne Twister plays a similar role to a seed in the generator used by C
- Demo: sim_mm1.py

```
# obtain setting and save it in a file
rand_state = random.getstate()
pickle.dump( rand_state, open( "rand_state_mm1.p", "wb" ) )
```

```
# load the saved setting and apply it
rand_state = pickle.load( open( "rand_state_mm1.p", "rb" ) )
random.setstate(rand_state)
```

Random number generators in Python

- Although both the random and numpy.random libraries use the Mersenne Twister generator, the generator for the libraries are separate

- The numpy.random library
  - You can generate an array of random numbers
  - The functions to get and set the state are numpy.random.get_state() and numpy.random.set_state()
  - Fewer distributions compared to the random library

Summary

- Basic concepts on pseudo-random number generators
- Using the inverse transform method to produce random numbers of different probability distributions
- Reproducibility – why and how

References

- Generation of random numbers
  - Raj Jain, “The Art of Computer Systems Performance Analysis”
    - Sections 26.1 and 26.2 on LCG
    - Section 28.1 on the inverse transform methods